
ATTRIBUTE-DRIVEN DESIGN DOCUMENT

for

LKFORUM WEBSITE

Version 1.4

Prepared by : Quach Gia Kiet

Submitted to : Nguyen Trinh Dong
Lecturer

May 16, 2026

Contents

1	Design Constraints	3
2	Quality Attribute Requirements	4
2.1	Performance	4
2.1.1	High-Throughput Content Retrieval	4
2.1.2	Asynchronous Processing	4
2.2	Security	5
2.2.1	Authentication and Authorization	5
2.2.2	Abuse Prevention with Rate Limiting	5
2.2.3	Data Privacy for Private Messaging	6
2.3	Supportability	6
2.3.1	Audit and User Activity Logging	6
2.3.2	Structured Error Diagnostics with slog	7
2.4	Manageability	7
2.4.1	Health and Readiness Checks	7
2.4.2	Runtime Metrics for Monitoring	8
2.5	Maintainability	8
2.5.1	Layered Architecture and Interface-Based Design	8
3	Architectural Representation	9
3.1	Logical View	9
3.2	Implementation View	11
3.3	Deployment View	12
3.4	Data View	14

1 Design Constraints

- **Performance:** The system must deliver fast content browsing and responsive real-time interactions. It uses the **Go/Gin** framework for high-concurrency request handling, **Redis caching** to offload repetitive read queries, and an **Event-Driven** architecture using in-memory **Go Channels** for non-blocking background tasks.
- **Security:** The system must protect accounts, privileged operations, and private conversations. It uses **JSON Web Tokens (JWT)** for authentication, **Role-Based Access Control (RBAC)** for authorization, Redis-backed **rate limiting** for abuse prevention, and client-side encryption for private messaging.
- **Supportability:** The system must make incidents and sensitive actions traceable. It uses structured runtime logging with Go **slog** for diagnostics and a dedicated audit-log model for critical user, moderator, and administrator actions.
- **Manageability:** The system must expose operational signals that allow administrators and developers to check service health and observe runtime behavior. It provides **/health**, **/ready**, and Prometheus-compatible **/metrics** endpoints.
- **Maintainability:** The architecture must remain easy to modify as LKForum grows. It enforces a **Layered Architecture** (Routes, Controllers, Services, Repositories) and interface-based boundaries to keep business logic, transport logic, and data access isolated.

2 Quality Attribute Requirements

2.1 Performance

2.1.1 High-Throughput Content Retrieval

Element	Statement
Stimulus	Thousands of users simultaneously request the global public feed.
Stimulus source	Guest Users, Members.
Environment	Peak usage times causing high read traffic.
Artifact	Post Service, Redis Cache Layer, MongoDB.
Response	The system serves feeds directly from Redis RAM using a Read-Through caching strategy, bypassing the database completely for cached endpoints.
Response measure	Database CPU utilization remains $\leq 60\%$ during peak loads. $\geq 95\%$ of feed read requests process within 500ms.

2.1.2 Asynchronous Processing

Element	Statement
Stimulus	A user publishes a new post that requires notifying hundreds of followers and updating analytics.
Stimulus source	Members.
Environment	Normal operation during content creation.
Artifact	Post Service, Notification Service, In-Memory Event Bus (Go Channels).
Response	The Post Service saves the content and immediately returns success to the user. It then asynchronously publishes an event to the Event Bus via Go Channels. Background goroutines consume the event and process notifications.
Response measure	The HTTP response time for post creation remains ≤ 1 second, unaffected by the number of followers receiving notifications.

2.2 Security

2.2.1 Authentication and Authorization

Element	Statement
Stimulus	Users attempt to access restricted API endpoints, such as deleting a post or changing community settings.
Stimulus source	Malicious Actors, Members, System Admins.
Environment	Normal operation over public internet.
Artifact	JWT Validation Middleware, RBAC Module.
Response	The system validates JWT signatures and strictly enforces RBAC rules. It checks if the authenticated user holds the required role (e.g., Moderator or Admin) before allowing the action.
Response measure	100% of unauthorized API requests are rejected with HTTP 403 Forbidden.

2.2.2 Abuse Prevention with Rate Limiting

Element	Statement
Stimulus	A user or malicious actor repeatedly calls sensitive endpoints such as login, OTP verification, post creation, or report submission.
Stimulus source	Members, Guests, Malicious Actors.
Environment	Public internet during normal operation or abuse attempts.
Artifact	Redis Rate Limiting Middleware, Auth Routes, Post Routes.
Response	The system increments a Redis counter scoped by endpoint and client identifier. Requests within the configured limit proceed normally; requests above the limit are rejected before reaching business logic.
Response measure	Excessive requests are rejected with HTTP 429 Too Many Requests within the active rate-limit window.

2.2.3 Data Privacy for Private Messaging

Element	Statement
Stimulus	Two members exchange private messages through a one-to-one conversation.
Stimulus source	Members.
Environment	Direct messaging via WebSocket connections.
Artifact	Client-side Cryptography Module (Svelte), WebSocket Service, Messaging Service, MongoDB.
Response	The sender's client encrypts the message before sending it to the backend. In the current implementation scope, an AES-GCM client-side prototype demonstrates the E2EE data path. The backend stores and routes only ciphertext, nonce, algorithm, and key version metadata.
Response measure	Private message plaintext is not stored in MongoDB; database records contain encrypted payload fields instead of readable message content.

2.3 Supportability

2.3.1 Audit and User Activity Logging

Element	Statement
Stimulus	A system administrator needs to trace the action history of a suspected malicious user.
Stimulus source	System Administrators, Security Auditors.
Environment	Post-incident investigation.
Artifact	Audit Log Service, Audit Log Repository, MongoDB <code>audit_logs</code> Collection.
Response	The system records a persistent audit entry for critical state-changing actions, including actor, role, action, target type, target identifier, reason, IP address, user agent, metadata, and timestamp.
Response measure	Critical authentication, moderation, report, ban, and unban actions are retrievable from the audit log collection for investigation.

2.3.2 Structured Error Diagnostics with slog

Element	Statement
Stimulus	A user reports an unexpected HTTP 500 Internal Server Error when attempting to vote on a poll.
Stimulus source	Members.
Environment	Production environment during fault condition.
Artifact	Go slog Logger, Service Layer, Middleware Layer.
Response	The backend emits structured diagnostic logs with stable fields such as action, user id, target id, request path, and error details. User-facing responses remain generic when internal details are sensitive.
Response measure	Developers can identify the failing component and relevant context from structured logs without searching unstructured console output.

2.4 Manageability

2.4.1 Health and Readiness Checks

Element	Statement
Stimulus	A developer or deployment environment needs to determine whether the backend process is alive and whether its required dependencies are reachable.
Stimulus source	Developers, Administrators, Deployment Environment.
Environment	Local development, staging, or production runtime.
Artifact	Go Backend, MongoDB Client, Redis Client, <code>/health</code> and <code>/ready</code> Endpoints.
Response	The backend exposes a liveness endpoint for process health and a readiness endpoint that checks MongoDB and Redis connectivity before reporting the service as ready.
Response measure	<code>/health</code> returns HTTP 200 when the process is alive; <code>/ready</code> returns dependency status for MongoDB and Redis.

2.4.2 Runtime Metrics for Monitoring

Element	Statement
Stimulus	Developers and administrators need visibility into request volume, latency, cache behavior, WebSocket activity, message delivery, moderation decisions, and notification creation.
Stimulus source	Developers, Administrators.
Environment	Runtime operation under normal and peak usage.
Artifact	Metrics Middleware, Metrics Registry, Prometheus-compatible <code>/metrics</code> Endpoint, Prometheus Scrape Configuration.
Response	The backend records counters, gauges, and histograms and exposes them through a Prometheus-compatible endpoint. Prometheus can scrape the backend for operational analysis.
Response measure	Metrics such as HTTP request counts, request durations, feed cache hit/miss counts, active WebSocket connections, messages sent, and moderation decisions are visible from <code>/metrics</code> .

2.5 Maintainability

2.5.1 Layered Architecture and Interface-Based Design

Element	Statement
Stimulus	A developer needs to add a new cross-cutting capability such as feed caching, rate limiting, audit logging, metrics, or private-message encryption support.
Stimulus source	Development Team.
Environment	Development and maintenance phases.
Artifact	Routes, Controllers, Services, Repositories, Interfaces, Bootstrap Wiring.
Response	The backend separates HTTP routing, request handling, business rules, data access, and infrastructure wiring. New capabilities can be introduced in the appropriate layer with limited changes to unrelated modules.
Response measure	Architectural changes remain localized to relevant routes, middleware, services, repositories, or platform packages instead of requiring broad rewrites across the codebase.

3 Architectural Representation

To describe the architecture of the LKForum System, the following views are presented:

3.1 Logical View

This view presents the system's modular decomposition into functional components and subsystems. It aligns with the **System Context** of the C4 model.

System Context Diagram - LKForum

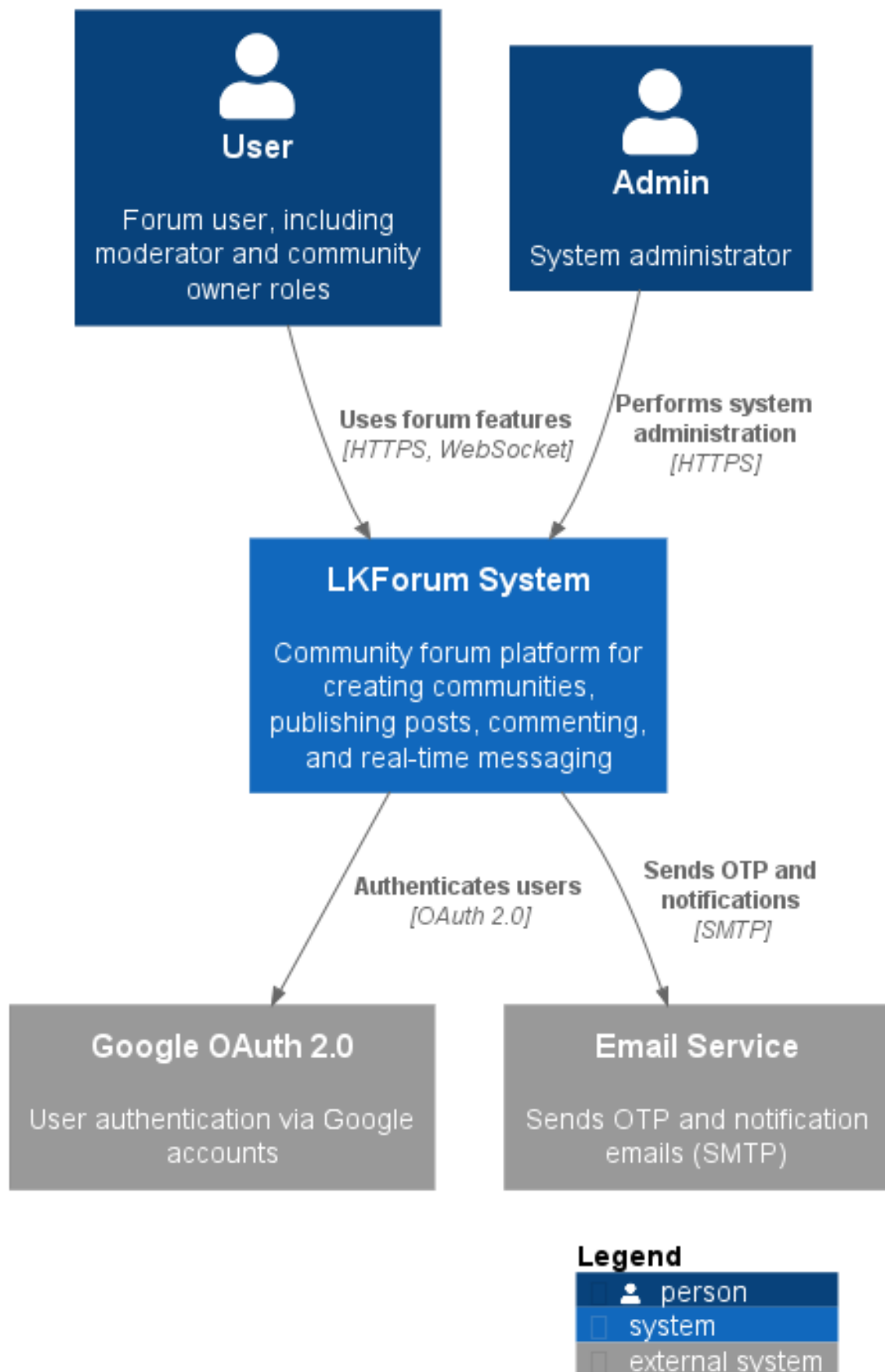


Figure 3.1: System Context Diagram

- **Layered Architecture & Interfaces:** The backend strictly separates concerns into Routes, Controllers, Services, and Repositories. Interfaces define contracts between layers (e.g., `PostService` interface), allowing for easy mock generation and unit testing.
- **IAM (Identity & Access Management) Module:** Handles user registration, JWT-based stateless authentication, Google OAuth SSO integration, and RBAC.
- **Community & Content Modules:** Manages the lifecycle of sub-forums, posts, and nested comments.
- **Event-Driven Hub:** An internal message bus utilizing **Go Channels and Goroutines** that decouples core business logic from auxiliary tasks (like firing push notifications or updating search indexes), ensuring high performance without relying on external message brokers.
- **Communication Module:** Facilitates real-time features including private messaging via WebSockets and client-side encrypted message payloads.

3.2 Implementation View

This view outlines the organization of code and deployment artifacts, aligning with the **Component Diagram** of the C4 model.

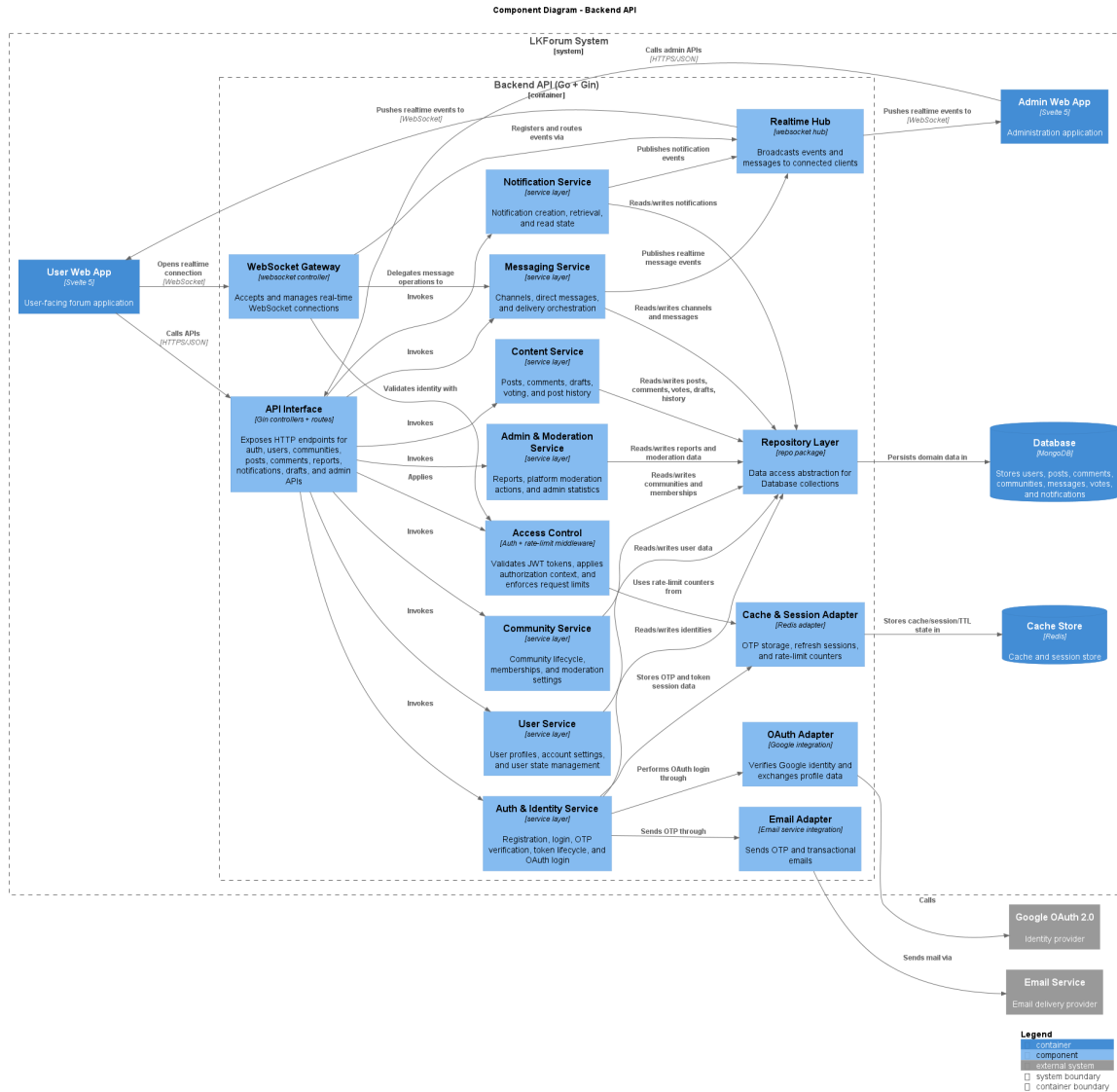


Figure 3.2: Backend Component Diagram

- **Backend Organization:** The backend is a monolithic Go application exposing REST APIs and WebSocket endpoints.
- **Frontend Organization:** Separated into two distinct SPAs built with Svelte and Vite:
 - **user-web:** The main interface for Guests, Members, and Community Moderators.
 - **admin-web:** A dedicated dashboard for System Administrators.
- **Observability Implementation:** The backend exposes Prometheus-compatible metrics, readiness checks, and structured runtime logs using Go `slog`. Critical business actions are stored separately as audit-log records.

3.3 Deployment View

Describes the physical deployment and runtime environment of system components, aligning with the **Container Diagram** of the C4 model.

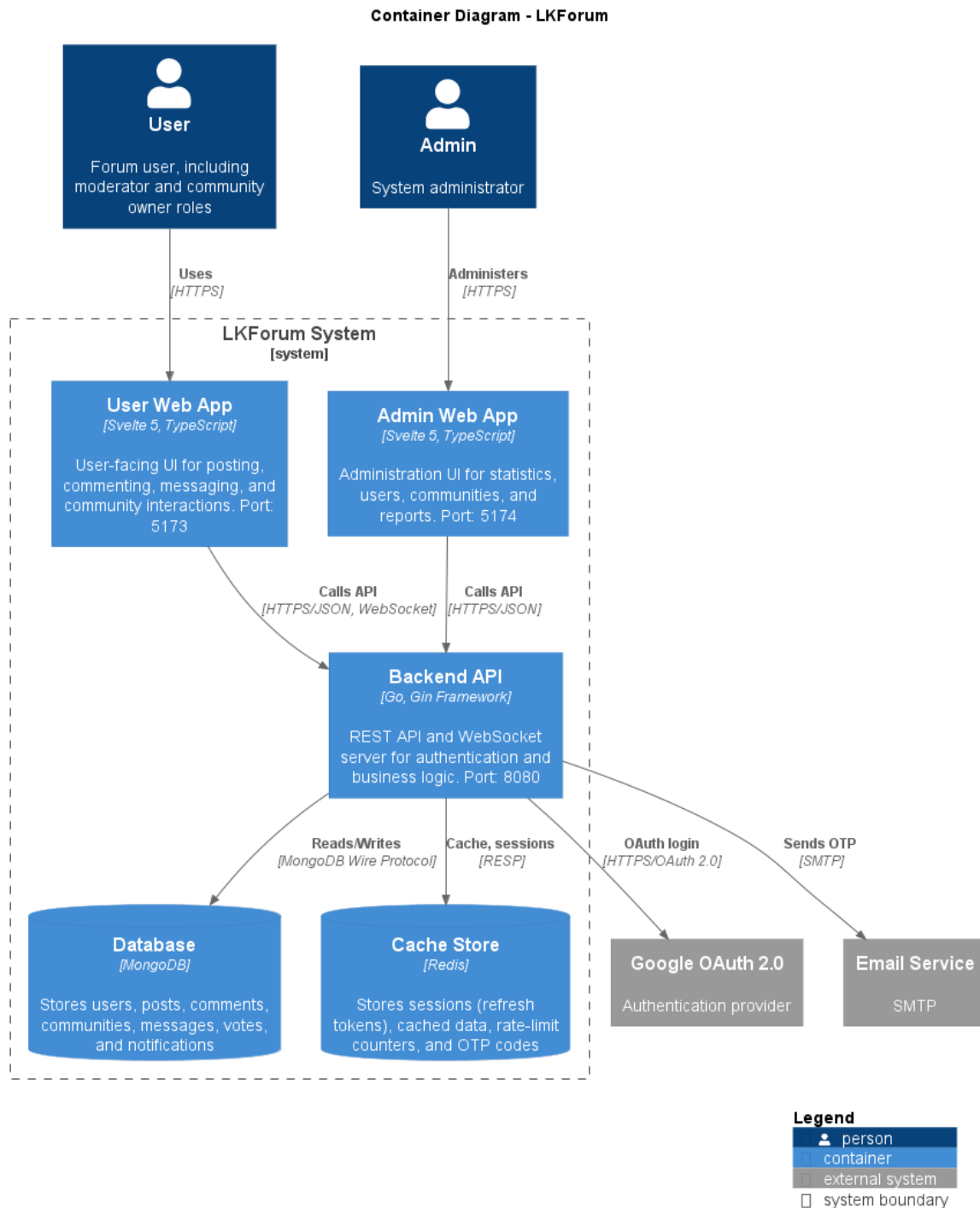


Figure 3.3: Container Diagram

- **Containerization:** The entire ecosystem is containerized using Docker, avoiding the overhead of Kubernetes while maintaining isolation.
- **Orchestration:** A `docker-compose.yml` file manages the multi-container application network.
- **Node Layout:**
 - **Application Nodes:** Go Backend Container, User Web Frontend Container, Admin Web

Frontend Container.

- **Data Nodes:** MongoDB container (persistent data), Redis container (caching).
- **Monitoring Nodes:** Prometheus container for scraping backend metrics.

3.4 Data View

Focuses on data models, storage strategy, and external data integrations.

- **Primary Storage (MongoDB):** Chosen for its flexible schema design, suitable for dynamic content types (polls, media arrays).
- **Caching (Redis):** Acts as an in-memory cache to absorb high read loads for public feeds, and stores fast-expiring data like session tokens and OTPs.
- **Secure Messaging Storage:** Private messages stored in MongoDB contain encrypted payload fields; the server routes them while client devices handle encryption and decryption in the current E2EE prototype.